

Time Travel und VACUUM bei Delta Lake

Section 1 · Databricks Intelligence Plattform · Quelle: Udemy-Kursmaterial „Advanced Delta Lake Features“, ergänzt mit Databricks-Dokumentation

Weil jede Änderung an einer Delta-Tabelle im Transaktionsprotokoll festgehalten wird (siehe vorheriges Kapitel), lässt sich der komplette Verlauf einer Tabelle nachvollziehen – und sogar rückgängig machen. Dieses Kapitel behandelt die drei praktischen Werkzeuge dafür: **Time Travel**, **RESTORE TABLE** und **VACUUM**.

1. Die Tabellenhistorie einsehen

Mit `DESCRIBE HISTORY` lässt sich der vollständige Audit-Trail einer Tabelle abfragen – jede Version mit Zeitstempel, ausführender Person, Operation und betroffenen Dateien:

```
DESCRIBE HISTORY my_table;
```

2. Time Travel: ältere Versionen abfragen

Auf Basis dieser Historie lässt sich jede frühere Version einer Tabelle gezielt abfragen – entweder über einen Zeitstempel oder über die fortlaufende Versionsnummer:

```
-- Per Zeitstempel
SELECT * FROM my_table TIMESTAMP AS OF "2019-01-01";

-- Per Versionsnummer (zwei gleichwertige Schreibweisen)
SELECT * FROM my_table VERSION AS OF 36;
SELECT * FROM my_table@v36;
```

Das ist besonders nützlich, um den Zustand einer Tabelle vor einer bestimmten Änderung zu vergleichen oder nachträglich zu prüfen, welche Werte zu einem früheren Zeitpunkt tatsächlich vorlagen.

3. Fehler rückgängig machen: RESTORE TABLE

Hat z. B. ein fehlerhafter Pipeline-Lauf versehentlich Datensätze gelöscht oder überschrieben, lässt sich die gesamte Tabelle mit einem einzigen Befehl auf einen früheren, bekannt guten Zustand zurücksetzen:

```
RESTORE TABLE my_table TO TIMESTAMP AS OF "2019-01-01";  
RESTORE TABLE my_table TO VERSION AS OF 36;
```

RESTORE ist dabei selbst nur eine weitere Transaktion: Es werden keine Dateien rückwirkend verändert, sondern anhand des Protokolls ermittelt, welche Dateien zur Zielversion gehörten, und diese Information als neuer, aktueller Log-Eintrag festgeschrieben – entsprechend schnell ist der Vorgang auch bei großen Tabellen.

4. Dateien kompaktieren und indizieren (Kurzüberblick)

Delta Lake kann viele kleine Dateien mit dem OPTIMIZE -Befehl zu größeren, effizienter lesbaren Dateien zusammenfassen (Compaction) und optional zusätzlich per ZORDER BY nach einer Spalte co-lokalisieren, um Data-Skipping-Algorithmen zu beschleunigen:

```
OPTIMIZE my_table;  
OPTIMIZE my_table ZORDER BY id;
```

Die Details zu Z-Ordering, Partitionierung und dem moderneren Liquid Clustering – inklusive der jeweiligen Vor- und Nachteile – behandelt **Section 6, Kapitel 2–3** dieser Dokumentation ausführlich.

5. Aufräumen: VACUUM

Da Delta Lake bei Updates und Löschungen alte Dateien nicht sofort physisch entfernt (siehe Copy-on-Write im vorigen Kapitel), sammeln sich mit der Zeit nicht mehr benötigte Dateien an – sowohl aus nicht mehr referenzierten alten Tabellenversionen als auch aus fehlgeschlagenen, unvollständigen Schreibvorgängen. Der VACUUM -Befehl entfernt genau diese verwaisten Dateien:

```
-- Alle Dateien entfernen, die älter als die Aufbewahrungsfrist sind (Standard: 7 Tage)  
VACUUM my_table;  
  
-- Explizite Aufbewahrungsfrist in Stunden angeben  
VACUUM my_table RETAIN 168 HOURS;
```

Die Standard-Aufbewahrungsfrist beträgt **7 Tage** – das ist bewusst so gewählt, damit noch laufende, länger dauernde Lesevorgänge nicht plötzlich auf gelöschte Dateien verweisen. Der entscheidende Merksatz für die Prüfung: **Nach einem VACUUM ist Time Travel für Versionen älter als die Aufbewahrungsfrist nicht mehr möglich** – die zugehörigen physischen Dateien existieren schlicht nicht mehr, unabhängig davon, was das Transaktionsprotokoll noch darüber weiß.

Ergänzt aus der offiziellen Databricks-Dokumentation: Die Aufbewahrungsfrist für `VACUUM` wird technisch über die Tabelleneigenschaft `delta.deletedFileRetentionDuration` gesteuert (Standardwert 7 Tage); Databricks rät ausdrücklich davon ab, diesen Wert zu unterschreiten. Getrennt davon regelt `delta.logRetentionDuration` (Standard 30 Tage), wie lange die JSON-Protokolldateien selbst aufbewahrt werden. Für `RESTORE TABLE` gilt: Die Wiederherstellung funktioniert nur, solange die zugehörigen Datendateien noch nicht durch ein `VACUUM` gelöscht wurden.

Quelle: [Remove unused data files with vacuum – Databricks-Dokumentation](#)